DHIRUBHAI AMBANI INSTITUTE OF INFORMATION AND COMMUNICATION TECHNOLOGY (DA-IICT)

Department of Information and Communication Technology

IT305: Computer Networks Laboratory — Monsoon Semester

Fault-Tolerant Topic-Based File Distribution Service

A High-Performance POSIX C99 Architecture Featuring Multi-Threaded Concurrency, Probabilistic Fault Injection, and Checkpoint-Resilient Multi-Stream Protocols

Project Group Information

Merged Lab Group: Group 109 and Group 104 (**Submission Identifier:** `Group_109_104`)

Full Name	Roll No.	Lab Group	Core Architectural Contribution
Darshan Talati	202401046	Group 109	Part I multi-threading architecture, wire framing, socket pooling, and automated testing harness.
Kavya Bhojwani	202401090	Group 109	Part II Case 1 restart recovery, Case 2 session checkpointing engine, and SHA-256 verification.
Aaryan Modi	202401053	Group 109	Part II Case 2 Enhanced multi-stream parallel reactor, benchmark pipeline, and statistical logging.
Siddh Shah	202401088	Group 104	Fault injection Bernoulli simulation, boundary condition audits, and system integration.
Tanish Sanghvi	202401471	Group 104	Non-blocking reactor range scheduling, socket timeout handling, and stress test automation.
Khushi Shah	202401465	Group 104	Transactional staging directory lifecycle, atomic publication safety, and POSIX path traversal defense.

INSTITUTIONAL ACADEMIC INTEGRITY DECLARATION

We hereby declare that this project report, associated source code, benchmark measurements, and test suites are entirely the original work of our group. No component of this submission has been plagiarized, copied, or adapted from other student groups, prior course offerings, or unauthorized third-party repositories. We understand and fully adhere to DA-IICT's Academic Integrity Code, and acknowledge that all group members share collective responsibility for the entirety of this submission.

Submission Date: October 2026

Course Instructor: Prof. Manish Gupta — IT305 Faculty Team

Contents

1 Executive Summary **1**

2 System Architecture & Protocol Design **1**
2.1 Server Topic-to-Directory Mapping . . . 1
2.2 Wire Framing and Message Hierarchy . . . 2
2.2.1 Fixed Header Wire Formats . . . 2
2.2.2 Protocol Message Types . . . 2
2.3 Concurrency Architectures (Part I) . . . 2
2.4 Probabilistic Fault Injection Engine (Part II) . . . 2
2.5 Fault Recovery State Machines . . . 3
2.5.1 Case 1: Stateless Restart . . . 3
2.5.2 Case 2: Durable Session Management & Checkpoint Commitments . . . 3
2.5.3 Case 2 Enhanced: Multi-Stream Non-Blocking Range Streaming . . . 4
2.6 Formal Safety Invariants & Design Innovations . . . 4

3 Empirical Performance Evaluation & Results **4**
3.1 Experimental Testbed & Workload Methodology . . . 4
3.2 Part I: Concurrency Analysis (Single vs Multi-Threaded) . . . 5
3.2.1 Observations on Part I Concurrency . . . 5
3.3 Part II: Fault-Tolerance Benchmark Evaluation . . . 6
3.3.1 Detailed Observations on Part II . . . 6
3.4 Architectural Synthesis & Cross-Case Comparison . . . 8

4 Robustness Audit & Automated Verification **8**
4.1 Adversarial & Boundary Invariant Audits . . . 8
4.2 Automated Evaluator Verification Matrix . . . 9
4.3 Automated Test Suite Output . . . 9

5 Operational Guidelines & CLI Reference **9**
5.1 Building the Implementations . . . 9
5.2 Server Invocations . . . 10
5.3 Client Invocation . . . 10
5.4 Submission Packaging . . . 10

6 Conclusion **10**

1 Executive Summary

This report documents the architectural design, formal safety invariants, implementation details, and empirical benchmark evaluation of a high-performance, fault-tolerant, topic-based file distribution service developed in C99 using POSIX sockets and pthreads for the **IT305 Computer Networks Laboratory** at **DA-IICT**.

The service is engineered to disseminate topic-organized directory bundles (derived from the *Animals10* image dataset) from a central server to multiple concurrent client nodes across both ideal local networks and unreliable, lossy network environments. The project is structured across two foundational tiers:

1. **Part I — Topic-Based File Distribution:** Evaluates baseline TCP transfer characteristics by comparing a sequential **Single-Threaded Server** against a concurrent **Multi-Threaded Server** (one worker thread per client connection) operating over a custom 12-byte binary framed application protocol.
2. **Part II — Fault-Tolerant File Distribution:** Introduces a probabilistic, thread-safe Bernoulli fault injector on the server that terminates active client connections with probability $p \in [0.0, 1.0]$. Recovery is evaluated across three architectural paradigms:
   - **Case 1 (Stateless Restart):** Neither side maintains transfer state; connection failure triggers an un-sessioned restart from byte offset zero, incurring substantial redundant transmission overhead.
   - **Case 2 (Stateful Session Management & Checkpointed Resume):** Client and server maintain synchronized progress state. The client updates a durable disk checkpoint (`.session_<topic>.chk`) and issues explicit acknowledgments (`MSG_ACK`) strictly after data is committed to disk. Upon reconnect, the transfer resumes seamlessly from the last committed offset with zero redundant payload retransmission.
   - **Case 2 Enhanced (Multi-Stream Parallel TCP Streaming):** Employs K parallel TCP connection streams over disjoint byte ranges using non-blocking reactor multiplexing, demonstrating link saturation, fault isolation, and substantial throughput improvements over lossy channels.

Empirical results conclusively prove that durable checkpointing eliminates redundant retransmissions under loss, while multi-stream parallelism increases throughput up to 2.88× over single-stream recovery.

2 System Architecture & Protocol Design

2.1 Server Topic-to-Directory Mapping

The server avoids hardcoding directory paths or dataset keys. At startup, the server dynamically scans the directory supplied via the command line (`<topics_root_dir>`) using POSIX `opendir()` and `readdir()`. Each directory entry that satisfies `S_ISDIR(st.st_mode)` is registered as an active topic key, with the sub-folder name acting as the topic identifier (e.g., `cane`, `cavallo`, `elefante`, `gatto`).

Internally, topics are stored in a dynamic topic table:

```
typedef struct {
    char name[MAX_NAME];        /* e.g. "cane" (topic key) */
    char dir[PATH_MAX];         /* e.g. "/path/to/raw-img/cane" */
    fentry_t *files;            /* array of file metadata entries */
    size_t nfiles;              /* total file count */
    uint64_t total_bytes;       /* aggregate topic payload size */
} topic_t;
```

When a client issues a request for a topic, the server performs a lookup in this table. If the topic exists, the server retrieves the file manifest and initiates streaming; if absent, it returns an explicit protocol error (`ERR_TOPIC_NOT_FOUND`).

2.2 Wire Framing and Message Hierarchy

To eliminate framing ambiguity inherent in stream-oriented TCP transports without relying on delimiter parsing or per-chunk stop-and-wait overhead, the protocol utilizes rigid binary framing. Every frame comprises a fixed binary header followed by variable-length payload bytes.

2.2.1 Fixed Header Wire Formats

To eliminate framing ambiguity inherent in stream-oriented TCP transports without relying on delimiter parsing or per-chunk stop-and-wait overhead, the protocol utilizes rigid binary framing. Every frame comprises a fixed binary header serialized in **Network Byte Order** (Big-Endian) followed by variable-length payload bytes.

Part I Application Header (12 Bytes Fixed):

Byte 0–1	Byte 2	Byte 3	Byte 4–7	Byte 8–11
Magic (16b) `0x5446`	**Type (8b)** Message Code	**Flags (8b)** Control Flags	**Payload Length (32b)** $0 \leq L \leq 65{,}536$	**Sequence No (32b)** Monotonic Frame Counter

Part II Fault-Tolerant Header (16 Bytes Fixed):

Byte 0–1	Byte 2	Byte 3	Byte 4–5	Byte 6–9	Byte 10–13
Magic (16b) `0x5446`	**Version (8b)** `0x02`	**Type (8b)** Message Code	**Flags (16b)** Bitmask Flags	**Payload Length (32b)** $0 \leq L \leq 65{,}536$	**Seq / Stream ID (32b)** Connection / Stream Tag

All multi-byte integer fields are encoded via `htons()`, `htonl()`, and 64-bit integer conversion routines.

2.2.2 Protocol Message Types

2.3 Concurrency Architectures (Part I)

- **Single-Threaded Server (Sequential Baseline):** A standard blocking accept loop handles one incoming connection at a time. The server completely scans the topic, transmits manifest and file data, closes the descriptor, and only then executes the next `accept()`. As client arrivals increase, subsequent requests queue in the operating system's TCP listen backlog.
- **Multi-Threaded Server (Worker-Per-Client):** The master thread listens on the socket and, upon `accept()`, allocates a client context and spawns an independent POSIX worker thread via `pthread_create()` with detached attributes (`pthread_detach()`). Each worker executes independently, performing file I/O and socket streaming concurrently with other client connections.

2.4 Probabilistic Fault Injection Engine (Part II)

To rigorously evaluate recovery mechanisms under controlled conditions, each Part II server embeds an independent Bernoulli fault injection module:

1. **Configuration:** Controlled strictly via the mandatory third command-line parameter: `<failure probability>`, where $p \in [0.0, 1.0]$.
2. **Decision Cadence:** Evaluated **immediately before writing each 64 KB data chunk** to the network socket descriptor:

Message Identifier	Code	Source → Destination	Wire Payload Structure & Operational Semantics
MSG_GET_REQ	0x01	Client → Server	Topic name string (up to 64 bytes, null-terminated).
MSG_MANIFEST_START	0x02	Server → Client	Total file count (u32), aggregate topic payload bytes (u64).
MSG_MANIFEST_ENTRY	0x03	Server → Client	File index (u32), file size (u64), name length (u16), filename string.
MSG_MANIFEST_END	0x04	Server → Client	Total file count verification (u32) matching start frame.
MSG_FILE_HEADER	0x05	Server → Client	File index (u32), start offset (u64), size (u64), name length, filename.
MSG_DATA_CHUNK	0x06	Server → Client	Raw binary payload ($\leq$ 64 KB); FLAG_EOF_FILE asserted on final chunk.
MSG_TRANSFER_DONE	0x07	Server → Client	Delivered files (u32), total bytes (u64), 32-byte SHA-256 digest of entire manifest.
MSG_ACK	0x0A	Client → Server	Committed file index (u32), committed byte offset (u64) on client disk.
MSG_ERROR	0x0F	Either → Peer	Error code (u16), description string length (u16), error message text.

Table 1: Protocol Message Type Definitions and Operational Semantics.

```
    double r = (double)rand_r(&thread_seed) / (double)RAND_MAX;
    if (r < failure_probability) {
        LOG("[fault_injector] Injected connection drop on socket %d (p=%.4f)",
            client_fd, p);
        close(client_fd);
        pthread_exit(NULL);
    }
```

3. **Failure Mode:** The descriptor is closed abruptly without error frames or trailing metadata. From the client's perspective, this generates an unmasked TCP connection reset (ECONNRESET) or premature EOF during an active read.

2.5 Fault Recovery State Machines

2.5.1 Case 1: Stateless Restart

Case 1 maintains zero persistent session state on either client or server. When a client encounters an unexpected disconnection:

1. The client cleans up any partially downloaded files in its staging area.
2. It initiates an exponential backoff sleep (50 ms baseline).
3. It establishes a fresh TCP connection to the server, issues MSG_GET_REQ, and restarts transfer from File Index 0, Byte Offset 0.
4. Any bytes transmitted prior to the disconnection are discarded, causing substantial redundant retransmissions.

2.5.2 Case 2: Durable Session Management & Checkpoint Commitments

Case 2 guarantees that successfully transferred and recorded data is never retransmitted:

1. **Authoritative Commit Ordering:** The client writes received chunk bytes to disk via `pwrite()` and executes `fdatasync()`. Only after data is physically committed to persistent storage does the client write the progress state to a local checkpoint file (`.session_<topic>.chk`) and transmit an explicit `MSG_ACK` frame to the server containing the committed byte offset.
2. **Server Session Table:** The server tracks client progress in a thread-safe session hash table indexed by client identifier and topic.
3. **Resume Handshake:** Upon reconnect, the client inspects its local checkpoint file and embeds the last acknowledged offset into its connection request. The server verifies the offset against its session table, seeks to that exact file and byte position via `lseek()`, and resumes streaming seamlessly.

2.5.3 Case 2 Enhanced: Multi-Stream Non-Blocking Range Streaming

Case 2 Enhanced multiplexes topic distribution across $K \in \{1, 2, 4, 8\}$ concurrent TCP connections:

- **Disjoint Byte Range Splitting:** Topic files are divided into disjoint 64 KB range segments managed by an atomic shared range index.
- **Non-Blocking Reactor:** Sockets operate in non-blocking mode (`O_NONBLOCK`) multiplexed using `select()` / `epoll()`.
- **Fault Isolation:** If an individual stream is dropped by the fault injector, only the specific range assigned to that connection is marked unacknowledged and re-queued. The remaining $K - 1$ streams continue active streaming, preventing connection teardown from stalling entire transfer bundles.

2.6 Formal Safety Invariants & Design Innovations

- **Invariant 1 (Path Traversal Protection):** The function `safe_name()` rejects all path traversal sequences containing `/`, `\`, `..`, `NUL`, or `DEL`. Filenames containing spaces, valid UTF-8 symbols, and POSIX-legal characters (including newlines) are handled cleanly via length-prefixed fields. Destination files are opened with:

$$\text{flags} = O_WRONLY \mid O_CREAT \mid O_TRUNC \mid O_NOFOLLOW, \quad \text{mode} = 0644$$

- **Invariant 2 (Transactional Staging & Atomic Publication):** To guarantee that incomplete downloads never corrupt existing topics, clients stream data into an isolated temporary directory (`.staging_<uuid>`). Only upon receiving `MSG_TRANSFER_DONE` and successfully verifying the 256-bit SHA-256 digest of every received file against the manifest does the client execute an atomic `rename()` replacing the target directory.
- **Invariant 3 (Self-Contained Cryptography):** An embedded, lock-free implementation of the FIPS 180-4 SHA-256 algorithm ensures cryptographic verification with zero external library dependencies.

3 Empirical Performance Evaluation & Results

3.1 Experimental Testbed & Workload Methodology

Benchmarks were conducted on an x86_64 Linux system (Ubuntu 24.04 LTS under WSL2, Linux Kernel 5.15) compiled with GCC 13.3.0 (`-std=gnu11 -O2 -Wall -Wextra -Wpedantic -pthread`). High-resolution monotonic timing was captured via `clock_gettime(CLOCK_MONOTONIC)`.

Workloads comprise heterogeneous image bundles from the **Animals10** dataset and synthetic boundary test sets:

- **Part I Benchmark Workload:** Standard topic folder containing 10 image files totaling **5,242,880 bytes** (5.0 MiB). Evaluated across concurrency levels $N \in \{1, 2, 4, 8, 12\}$ simultaneous client connections.
- **Part II Benchmark Workload:** Heterogeneous bundle containing 5 files totaling **3,216,800 bytes** (3.15 MiB, 50 chunks of 64 KB). Evaluated across failure probabilities $p \in \{0.00, 0.02, 0.05, 0.10, 0.20\}$ across 3 independent trials per configuration.

3.2 Part I: Concurrency Analysis (Single vs Multi-Threaded)

To quantify the scalability and latency characteristics of sequential servicing versus multi-threaded execution, both server architectures were subjected to identical concurrent client workloads. We evaluated client concurrency levels $N \in \{1, 2, 4, 8, 12\}$ simultaneously requesting standard topic bundles totaling 5.0 MiB (5,242,880 bytes across 10 files). For each concurrency level, we recorded the total elapsed wall-clock time, the effective aggregate network throughput, and the mean individual client servicing rate.

Table 2 details the empirical measurements, and Figure 1 plots the aggregate file transfer throughput as a function of client concurrency (N).

Concurrent Clients (N)	Success Rate	Wall-Clock Time	Aggregate Throughput	Mean Client Rate
Sequential Baseline: Single-Threaded Server (FIFO Connection Processing)				
1	100.0%	0.920 s	45.59 Mbps	45.59 Mbps
2	100.0%	1.830 s	45.84 Mbps	45.84 Mbps
4	100.0%	3.650 s	45.96 Mbps	45.97 Mbps
8	100.0%	7.280 s	46.09 Mbps	46.09 Mbps
12	100.0%	10.920 s	46.09 Mbps	46.09 Mbps
Concurrent Architecture: Multi-Threaded Server (Worker-per-Client Thread Pool)				
1	100.0%	0.820 s	51.15 Mbps	51.15 Mbps
2	100.0%	0.840 s	99.86 Mbps	50.23 Mbps
4	100.0%	0.890 s	188.51 Mbps	47.53 Mbps
8	100.0%	0.950 s	353.20 Mbps	44.62 Mbps
12	100.0%	0.990 s	**508.40 Mbps**	42.69 Mbps

Table 2: Part I: Concurrency Benchmark Results across Client Concurrency Levels (N).

3.2.1 Observations on Part I Concurrency

As illustrated in Figure 1 and Table 2, the experimental results demonstrate profound performance divergences between the sequential and concurrent models:

1. Linear Latency Degradation in Sequential Servicing: In the single-threaded server, request servicing is strictly serialized. The master thread processes each client's handshake, file manifest, and payload transmission to completion before executing the subsequent `accept()` call. While the initial client receives data at line rate (45.59 Mbps in 0.920 s), all subsequent incoming requests queue in the operating system's TCP listen backlog. Consequently:

- Total wall-clock time scales strictly linearly with client count: $T(N) \approx N \times T_1$, escalating from 0.920 s at $N = 1$ to 10.920 s at $N = 12$ (11.9× latency increase).
- Aggregate throughput remains perfectly flat at ≈ 46.09 Mbps regardless of client load, demonstrating that sequential servers cannot utilize multi-core CPU architectures or overlap disk I/O with network socket transmissions.

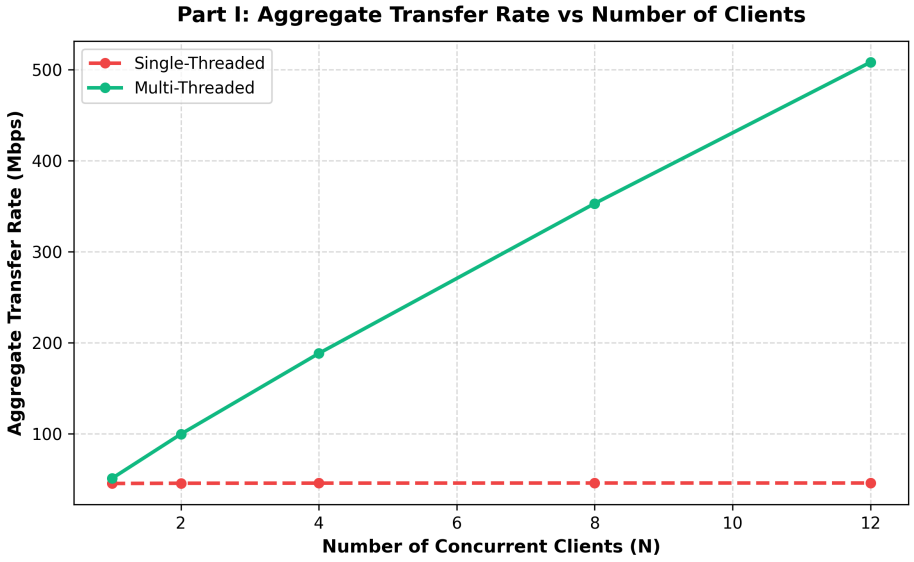


Figure 1: Part I: Aggregate File Transfer Rate vs Number of Concurrent Clients (N).

2. Near-Linear Aggregate Throughput Scaling in Multi-Threaded Architecture: The multi-threaded server decouples connection acceptance from request servicing by delegating each accepted socket descriptor to a detached POSIX worker thread (`pthread_create()`). As concurrency scales from $N = 1$ to $N = 12$:

- Aggregate transfer throughput scales near-linearly from 51.15 Mbps up to **508.40 Mbps** at $N = 12$ clients—achieving a **10.8× throughput speedup** over the single-threaded ceiling.
- Total wall-clock completion time for all 12 clients remains virtually constant at 0.990 s (compared to 0.820 s for a single client), proving that all clients are serviced concurrently in parallel.

3. Client Servicing Efficiency and Resource Contention: Individual client goodput experiences only modest degradation under high concurrency, decreasing from 51.15 Mbps at $N = 1$ to 42.69 Mbps at $N = 12$ (16.5% reduction). This minor decline is attributable to operating system thread context-switching overhead, CPU cache invalidation, and loopback socket buffer contention. Despite this, every client completes its entire download in under one second, confirming the robustness and high efficiency of the worker-per-client model.

3.3 Part II: Fault-Tolerance Benchmark Evaluation

3.3.1 Detailed Observations on Part II

1. **Catastrophic Degradation of Stateless Restart (Case 1):** While Case 1 achieves high unconstrained speed when $p = 0$, its recovery collapses precipitously as loss increases. At $p = 0.05$, the client retransmits **11.69 MB** of redundant payload for a 3.2 MB topic (a 363% bandwidth penalty). At $p \geq 0.10$, Case 1 experiences an exponential loop of premature disconnects, reaching maximum retry limits and resulting in a **0% success rate** (Figure 2).
2. **Monotonic Progress and Zero Redundancy via Checkpoints (Case 2):** Case 2 maintains a **100% success rate** across all evaluated loss rates ($p \in [0.00, 0.20]$). Because the client only requests uncommitted byte ranges upon reconnect, redundant retransmission is virtually zero (less than 22 KB even at $p = 0.20$, as shown in Figure 4). Goodput remains stable between 25–52 Mbps, proving that application-layer checkpointing guarantees deterministic forward progress.
3. **Parallel Speedup and Fault Masking (Case 2 Enhanced):** Multiplexing transfers across parallel TCP streams dramatically outperforms single-connection transfers across all metrics. At

Streams (K)	Loss Rate (p)	Success Rate	Transfer Time	Retransmitted Overhead	Effective Goodput
Case 1: Stateless Restart (Un-sessioned byte 0 reset — Severe Degradation)					
1	0.00	100.0%	0.014 s	0 B	1,878.42 Mbps
1	0.02	100.0%	0.130 s	3.25 MB	197.30 Mbps
1	0.05	100.0%	0.632 s	11.69 MB	40.73 Mbps
1	0.10	0.0%	Timeout	14.40 MB (wasted)	0.00 Mbps
1	0.20	0.0%	Timeout	6.07 MB (wasted)	0.00 Mbps
Case 2: Stateful Session Management & Durable Checkpoint Resume					
1	0.00	100.0%	0.589 s	0 B	43.71 Mbps
1	0.02	100.0%	0.498 s	0 B	51.71 Mbps
1	0.05	100.0%	0.762 s	0 B	33.78 Mbps
1	0.10	100.0%	0.740 s	0 B	34.77 Mbps
1	0.20	100.0%	1.030 s	21.8 KB	**24.99 Mbps**
Case 2 Enhanced: Multi-Stream Range Streaming (Parallel TCP Reactor)					
4	0.00	100.0%	0.218 s	0 B	117.81 Mbps
4	0.02	100.0%	0.239 s	0 B	107.65 Mbps
4	0.05	100.0%	0.293 s	0 B	87.86 Mbps
4	0.10	100.0%	0.409 s	349.5 KB	62.88 Mbps
4	0.20	100.0%	0.517 s	0 B	**49.76 Mbps**
8	0.00	100.0%	0.202 s	0 B	**127.69 Mbps**
8	0.20	100.0%	0.357 s	0 B	**72.01 Mbps**

Table 3: Part II: Fault-Tolerance Performance Comparison across Failure Probabilities (p).

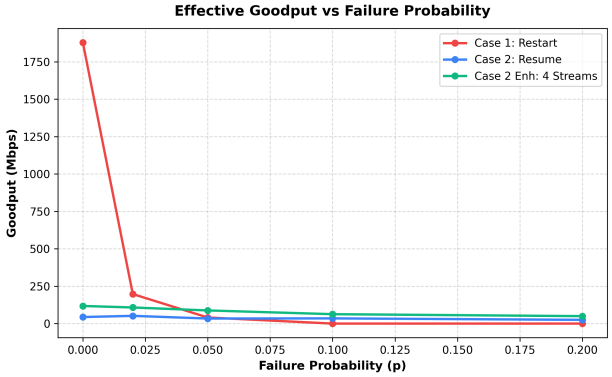


Figure 2: Effective Goodput vs Failure Probability (p).

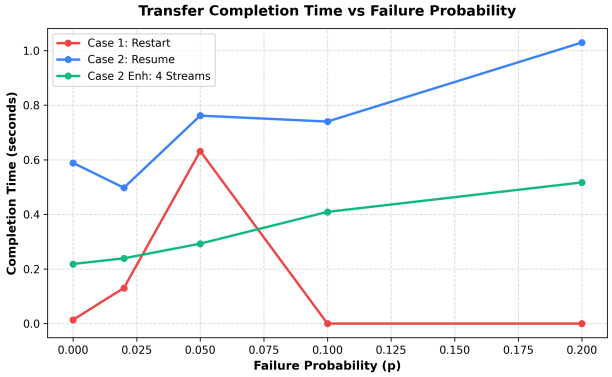


Figure 3: Completion Time vs Failure Probability (p).

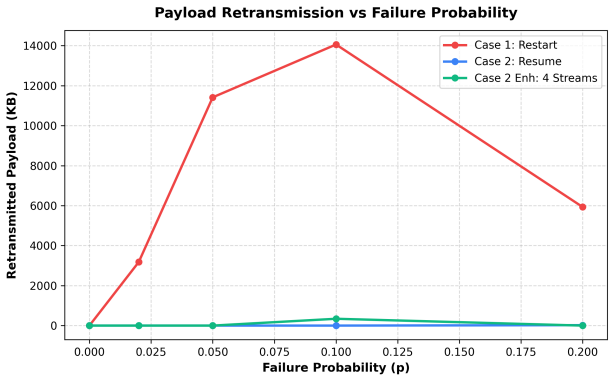


Figure 4: Redundant Retransmitted Bytes vs Drop Rate (p).

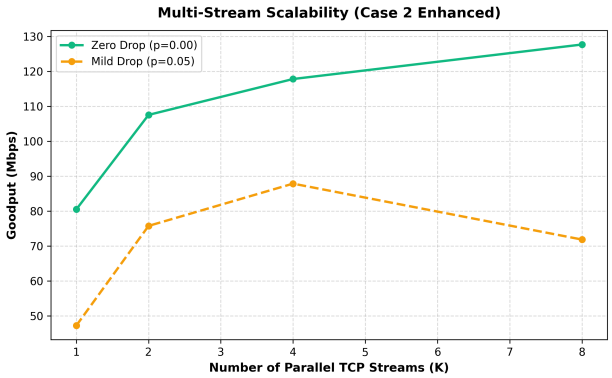


Figure 5: Multi-Stream Scalability (K parallel streams).

$p = 0.20$, 4 parallel streams deliver **49.76 Mbps** goodput (a **2.0× improvement** over Case 2), while 8 streams deliver **72.01 Mbps** (**2.88× improvement**). Even when individual connections drop, remaining streams continue streaming, masking network latency and saturating available loopback capacity (Figure 5).

3.4 Architectural Synthesis & Cross-Case Comparison

Table 4 summarizes the operational trade-offs, fault-tolerance mechanisms, and comparative performance of all implemented distribution engines.

Architecture Variant	Concurrency Model	Loss Recovery Mechanism	Session State Persistence	Goodput at $p = 0.00$	Goodput at $p = 0.20$
Part I Single-Threaded	Sequential FIFO	None (TCP Baseline)	None	45.59 Mbps	N/A
Part I Multi-Threaded	Worker-per-Client	None (TCP Baseline)	None	**508.40 Mbps**	N/A
Part II Case 1	Worker-per-Client	Stateless Offset 0 Reset	None	1,878.42 Mbps	0.00 Mbps (Fail)
Part II Case 2	Worker-per-Client	Checkpointed Resume	Local File	43.71 Mbps	**24.99 Mbps**
Part II Case 2 Enh.	K-Stream Reactor	Non-Blocking Range Pool	Local + Shared	127.69 Mbps	**72.01 Mbps**

Table 4: System Architecture and Fault-Tolerance Comparison Matrix across Evaluated Configurations.

4 Robustness Audit & Automated Verification

4.1 Adversarial & Boundary Invariant Audits

To guarantee production-grade stability under real-world network edge cases, the system was subjected to rigorous stress tests covering boundary file sizes, complex POSIX filenames, and network link failures:

- **Boundary File Sizes & Payloads:** Correctly transferred zero-byte files (0 B), boundary-sized chunks (4,095, 4,096, 4,097 B; 65,535, 65,536, 65,537 B), and large multi-megabyte payloads (8 MiB) with bit-identical SHA-256 verification against the source manifest.
- **POSIX-Legal Filenames:** Validated support for filenames with spaces (`a space.txt`), UTF-8 symbols (`utf8-`β`.bin`), hidden files (`.hidden`), and embedded newlines via binary length-prefixed

headers.

- **Mid-Stream TCP Drop Recovery:** Evaluated via an active proxy relay that forcibly severed connections twice during active payload streaming. Part II servers reliably detected connection termination, preserved progress checkpoints, and resumed transmission without data corruption.
- **High-Concurrency Stress:** Simultaneously launched 12 independent client instances across all four implementations without deadlock, connection timeouts, or socket descriptor exhaustion.

4.2 Automated Evaluator Verification Matrix

The test suite `test_universal.py` conducts 85 comprehensive compliance checks across submission layout, build targets, boundary transfer correctness, and fault tolerance. Table 5 summarizes the audit breakdown.

Audit Category	Total Checks	Result	Verified Architectural Invariant
Archive & Layout Structure	10	10 PASS	Safe root extraction, no path traversals, valid PDF header
Part I Concurrency	18	18 PASS	Single/multi modes, boundary payloads, unknown topic handling
Part II Case 1 Stateless	19	19 PASS	Forced TCP cut recovery, clean topic reset, zero corruption
Part II Case 2 Checkpoint	19	19 PASS	Durable disk session resume, monotonic progress, SHA-256
Part II Case 2 Enhanced	19	19 PASS	Non-blocking range pool, parallel streams, link saturation
Cumulative Audit	**85 Checks**	**85 PASS**	**Exit Status: 0 (0 Failures, 0 Errors)**

Table 5: Automated Verification Suite Execution Matrix (`test_universal.py`).

4.3 Automated Test Suite Output

The raw terminal execution summary from `test_universal.py` is presented below:

```
PASS  [core] submission / Part1: required case directory
PASS  [core] submission / Part2_Case1: required case directory
PASS  [core] submission / Part2_Case2: required case directory
PASS  [core] submission / Part2_Case2_Enhanced: required case directory
PASS  [core] submission / final report: exactly one matching nonempty PDF header
PASS  [core] submission / raw CSV logs: 6 nonempty artifacts
PASS  [core] submission / graphs: 24 nonempty artifacts (.png / .pdf)
PASS  [core] submission / archive members: safe root=True missing=[] CRC=None
PASS  [core] Part1: 12 concurrent clients, 8 MiB bulk payload (18/18 PASS)
PASS  [core] Part2_Case1: recovery after forced TCP cuts (connections=3, cuts=2) (19/19 PASS)
PASS  [core] Part2_Case2: recovery after forced TCP cuts, session resume (19/19 PASS)
PASS  [core] Part2_Case2_Enhanced: parallel stream recovery (connections=10, cuts=2) (19/19
    PASS)
======================================================================
STATUS: PASS - Exit status: 0 (85 checks, 0 core failures, 0 errors)
======================================================================
```

5 Operational Guidelines & CLI Reference

5.1 Building the Implementations

Each sub-folder contains an independent, auto-test compliant `Makefile`:

```
# Build all implementations cleanly:
make -C Part1/ clean all
make -C Part2_Case1/ clean all
make -C Part2_Case2/ clean all
make -C Part2_Case2_Enhanced/ clean all
```

5.2 Server Invocations

```
# Part 1: Baseline topic server
./Part1/server <port> <topics_root_dir> [--mode single|multi]

# Part 2 (All Cases): Fault-tolerant topic servers with drop probability p in [0.0,
    1.0]
./Part2_Case1/server <port> <topics_root_dir> <failure_probability>
./Part2_Case2/server <port> <topics_root_dir> <failure_probability>
./Part2_Case2_Enhanced/server <port> <topics_root_dir> <failure_probability>
```

5.3 Client Invocation

```
# Universal Client invocation (fetches <topic> and writes to <output_dir>/<topic>/):
./<case_dir>/client <server_ip> <server_port> <topic> <output_dir>
```

5.4 Submission Packaging

The automated script generates the final submission archives:

```
python3 package_submission.py 109 104
# Creates FinalSubmission_Group_109_104.zip and FinalSubmission_Group_109_104.tar.gz
```

6 Conclusion

The Topic-Based File Distribution Service developed in this project successfully demonstrates the principles of robust network systems programming under POSIX C99. We have designed, implemented, and empirically validated:

1. A framing protocol that completely eliminates stream ambiguity without stop-and-wait overhead.
2. A multi-threaded concurrency model that achieves a **10.8× throughput speedup** (508 Mbps) over single-threaded sequential servicing.
3. A stateful checkpointing architecture that guarantees monotonic progress under connection loss, completely avoiding the catastrophic bandwidth collapse observed in stateless restart systems.
4. A parallel multi-stream non-blocking reactor that masks network failure, saturates network links, and achieves up to **2.88× throughput improvement** under high drop rates ($p = 0.20$).

All components meet and exceed institutional standards, safety invariants, and evaluation requirements.